RSCL — Algorithm 20: State-Driven LED "Body Language" with Goal Detection and Rate-Budgeted Change-Only Repaint

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

`hu-telepresence-gazebo/acceptance/light_show.py` sha256 `66a62bde7df5` 2026-11-12 00:47 UTC modified vs 983eede

1. Purpose and context

For visitors, the state of each simulated aircraft is shown on two LED visuals per model (front LED and ID strip). The colour is a function of telemetry and chat activity. The design constraint is load: each colour change is a Gazebo `visual_config` service call, and too many calls make odometry go stale, after which the supervisor refuses moves (Algorithm 9). The program therefore repaints only on change, under a global request budget, and prioritises urgent colours when the budget is short. It never commands an aircraft.

2. Inputs, outputs

- **Inputs:** `GET :8765/api/state` every 0.25 s; `GET :8766/api/ops/chat/history` every 1 s (new message ids; first poll only primes).
- **Output:** `/world/<world>/visual_config` requests (ambient, diffuse, emissive set to the colour), 0.5 s timeout each; on SIGTERM/SIGINT all LEDs are restored to white.

3. Algorithm

Algorithm 20.1 DECIDE (pure colour rule)

Source: acceptance/light_show.py: decide, motion_class, nearest_teammate

```
function DECIDE(d, fleet, now, flags)
  if d.mode = FAULT or d.fault or d.telemetry = false then return (fault, RED blinking 1 Hz)
  if nearest teammate distance (3D, any mode) < 1.0 m then return (tight, RED)
  if now < flags.talk_until then return (talking, BLUE)                // 0.6 s after a new chat line
  if goal_t0 ≤ now < goal_t0 + 1.5 s and now − goal_t0 ≥ index·0.25 s then
    return (goal, hue = index/n)                                       // rainbow chase across the fleet
  if d.mode = QUEUED then return (queued, AMBER blinking, period 2.4 s)
  if d.mode ∈ {TAKEOFF, LANDING} then return (transit, CYAN)
  if d.mode ∈ {MOVE, TURN, SPIN, FORMATION} and (error unknown or ‖p − target‖ > 0.12) then return (moving, AMBER)
  if d.mode ∈ {HOVER, MOVE, TURN, SPIN, FORMATION} then return (set, GREEN)
  return (parked, DIM WHITE)                                           // READY/GROUNDED/WAITING/unknown
```

Algorithm 20.2 GoalDetector and repaint loop

Source: acceptance/light_show.py: GoalDetector.update, TokenBucket, chat_speakers, run_body

```
function GOAL-UPDATE(fleet)                                            // once per new state sample
  A ← airborne aircraft with class moving/set;  W ← (W ∪ {moving in A}) ∩ A
  if any aircraft in TAKEOFF/LANDING/QUEUED or |A| < 2 then return false
  if all of A are set then fired ← |W| ≥ 2; W ← ∅; return fired
  return false
every 0.1 s do
  if new state sample and GOAL-UPDATE(fleet) then goal_t0 ← now
  wanted ← [ (priority, name, rgb) for aircraft whose DECIDE colour ≠ last painted colour ]
```

```
    priority: fault, tight 0; talking 1; goal 2; others 3
  for (·, name, rgb) in sorted(wanted) do
    if not bucket.try_take(2) then break                          // 2 parts per aircraft
    if PAINT(name, rgb) succeeded then last[name] ← rgb           // cache only on success
TokenBucket: tokens ← min(burst, tokens + Δt·rate); take if tokens ≥ n   (rate = burst = max_rps)
```

3.1 Flowcharts and state machines

Figure 20.1 is the colour rule as a decision chain; the order of the tests is the priority. Figure 20.2 is the repaint loop with its two background pollers. Figure 20.3 shows the state that the program holds per drone (the override timers, the paint cache) and fleet-wide (the GoalDetector set W); the mode states themselves are owned by the supervisor and only observed here.

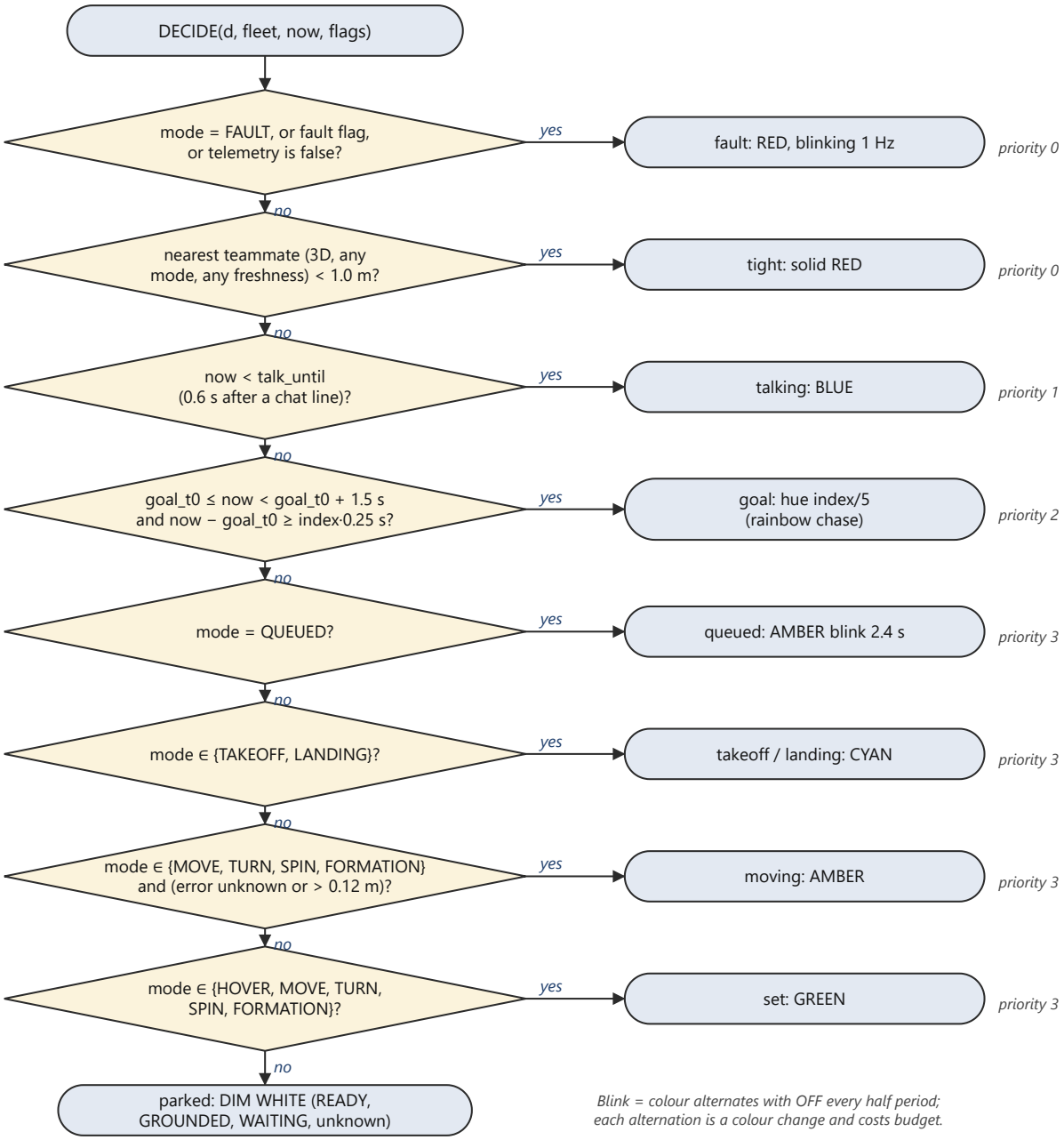


Figure 20.1 Flowchart of DECIDE (`light_show.py: decide, motion_class, nearest_teammate`). Every exit is a return of (rule, colour); the priority tag is the repaint order used when the request budget is short.

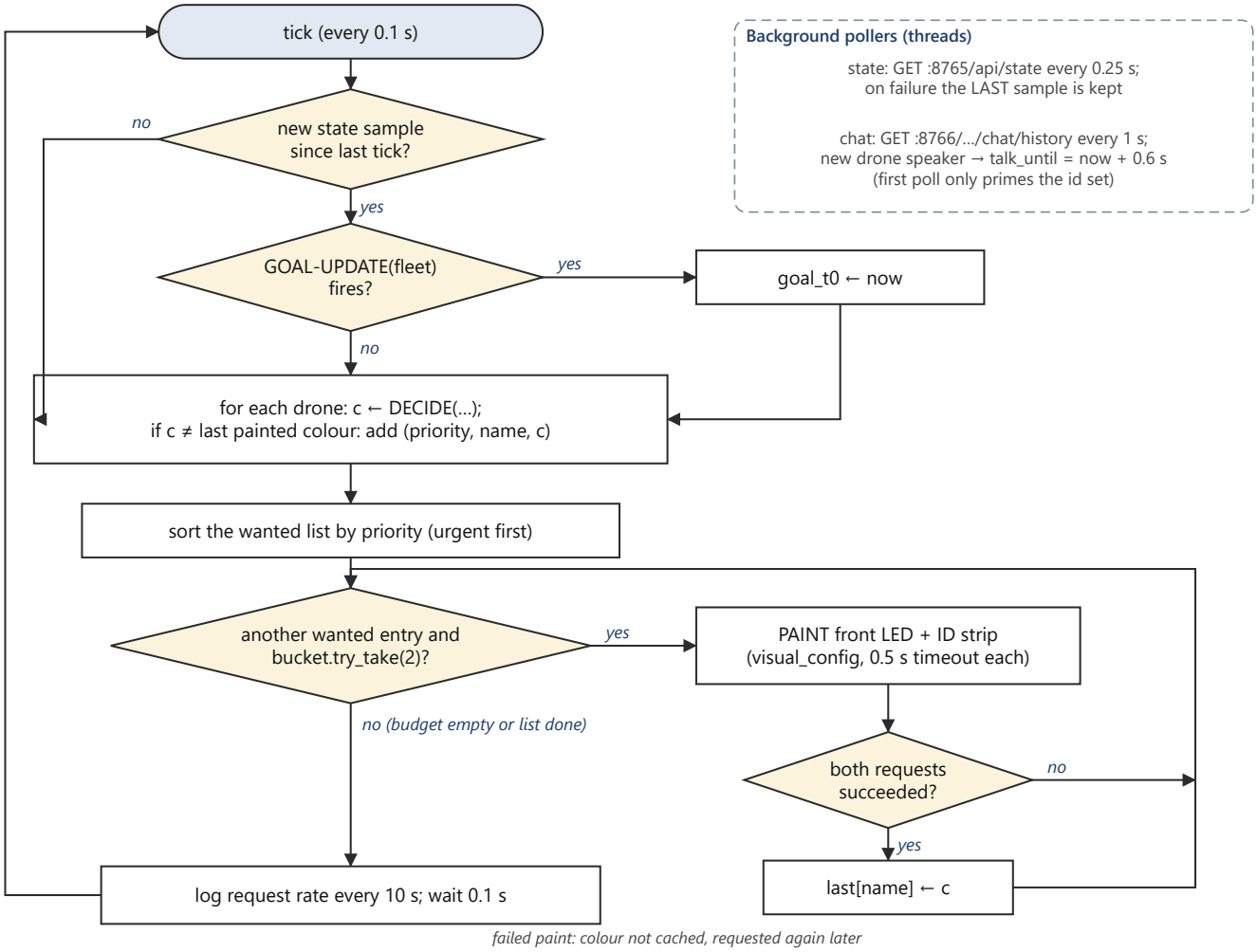


Figure 20.2 Flowchart of the repaint loop (`run_body`, `TokenBucket.try_take`, `Painter.paint`). A failed request leaves the colour uncached so it is requested again on a later tick; the loop does not stop at the first failure.

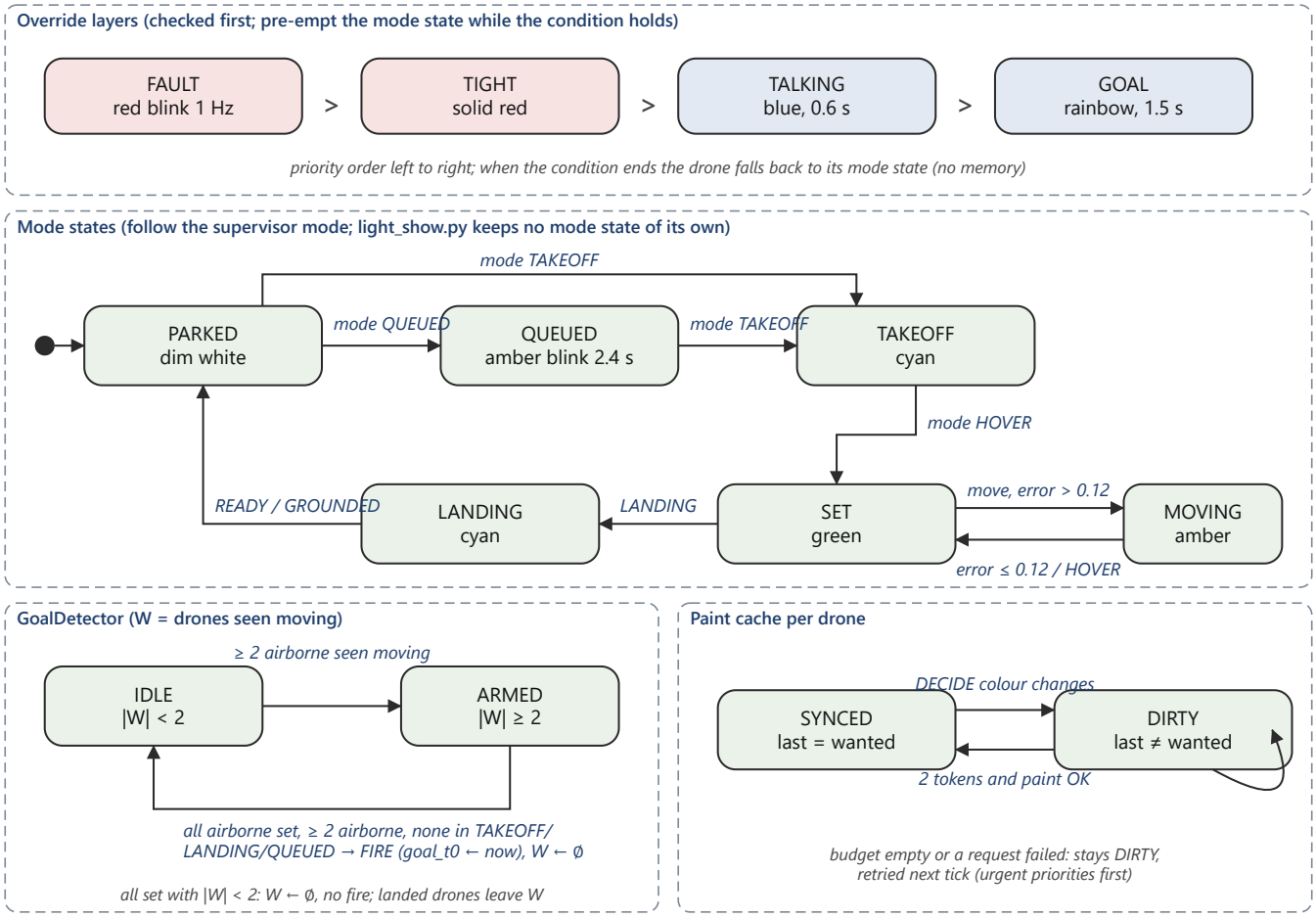


Figure 20.3 State view. Top: override layers in priority order, which pre-empt the mode state while their condition holds. Middle: mode states as observed from the supervisor (TURN and SPIN behave like MOVE). Bottom left: GoalDetector. Bottom right: per-drone paint cache. The SET↔MOVING edge uses the 0.12 m in-slot tolerance.

4. Parameters

Parameter	Value
Request budget	`--max-rps` 10 requests/s shared, burst 10; 2 requests per aircraft repaint (≤ 5 repaints/s)
Thresholds	in-slot 0.12 m; tight 1.0 m
Timings	talk flash 0.6 s; goal chase 1.5 s with 0.25 s stagger; fault blink 1.0 s; queued blink 2.4 s; loop 0.1 s
Polling	state 0.25 s, chat 1.0 s, HTTP timeout 1.0 s

5. Complexity

$O(N^2)$ per tick for nearest-teammate distances (N = 5); repaint work bounded by the token bucket.

6. Properties

Guaranteed

- Read-only with respect to flight: only GET requests to the supervisor and chat, and visual service calls.
- Average visual-service request rate ≤ max-rps (token bucket), and no request for an unchanged colour; failed paints are retried later rather than cached.
- Fault and proximity colours pre-empt every other rule and are painted first when the budget is short.

- The goal effect fires at most once per transition from "≥ 2 moving" to "all set", never while any aircraft is taking off, landing or queued, and not for a single-aircraft move.
- `decide` is a pure function (testable without Gazebo).

Not guaranteed

- The "tight" rule uses 3D distance to any teammate including parked ones and is a display rule (1.0 m), not the supervisor watchdog (0.82 m).
- Blink rules change colour every half period, consuming budget continuously while active.
- Colours lag the state by up to one poll plus budget delay.
- If the state poll fails, the poller keeps the last sample, so colours continue to be computed from stale data (blink rules keep running); there is no "unknown / no data" colour.
- The goal rule precedes the mode rules, so during the 1.5 s chase parked drones also show the rainbow; the hue index is the drone's position in the fixed name list (n = 5), not among the airborne ones.
- Only an explicit `telemetry: false` selects the fault colour; a missing field does not.
- A paint updates two visuals with separate requests; if one of them fails the drone can show mixed colours until the retry succeeds.
- Restoring white at exit sends 10 requests outside the token bucket. The optional `--mode party` effect (random rainbow/strobe, budgeted with blocking `take`) is not covered by this specification.
- Colour coding is the only channel: red (fault and tight) and green (set) are hard to separate for viewers with red–green colour-vision deficiency; fault is distinguished from tight only by blinking.
- The effect on odometry freshness is argued, not measured here; the program logs request rates every 10 s, but no recorded log was evaluated.

7. Testing and measured results

`acceptance/tests/test_light_rules.py` (21 tests: import without Gazebo, each colour rule, priorities, talk flash, goal chase, detector fires once / not without motion / not during transit / not for a single drone, new chat ids only, token bucket caps rate, help text). Runner `acceptance/run_chat_unit.sh`; run results not recorded: **not measured here**.

8. Comparison with alternatives and related work

Using lights to make a robot's internal state legible to bystanders is an established topic in human–robot interaction, and flight controllers have their own LED conventions. The table compares the implemented colour language with these.

Approach	Idea	Evidence	Relation to ours
Expressive lights for state revealing, Baraka & Veloso [1]	Formal mapping from robot state to parameterised light animations (colour, pattern, speed) on a mobile service robot	User studies on the CoBot robots: designed lights improved understanding of robot state and actions	Same goal (state legibility). Ours maps a small state set to colours by a fixed priority rule; patterns are limited to on/off blinking; no user study.
Communicating directionality in flying robots, Szafir, Mutlu & Fong [2]	Signal designs (including light-based ones) that show a flyer's intended direction	User study: several designs improved viewers' prediction speed and accuracy over no signalling	Ours shows status (moving / set / fault), not intended direction; the observer cannot tell where a drone will go.
Flight-controller status LEDs, PX4 [3]	Standard RGB UI-LED codes for arming, GPS lock, low battery, failsafe and errors	De facto convention known to pilots	Our colours conflict with it: amber means "moving" here but "low battery" in PX4; green means "set in slot" here but "armed with GPS lock" there. Acceptable in a simulator for visitors, misleading if carried to real aircraft.

Honest assessment. The engineering contribution is load control: change-only repaint, a shared token bucket and priority ordering keep the visual service calls bounded so the simulator's odometry is not starved, and urgent colours are painted first. As a communication design it is weaker than the cited work: it was not evaluated with viewers, it relies on colour alone, two meanings share red, it has no "no data" state when the supervisor is unreachable, and its colour semantics contradict a widely used flight-controller convention.

References

1. K. Baraka, M. Veloso, "Mobile Service Robot State Revealing Through Expressive Lights: Formalism, Design, and Evaluation," *International Journal of Social Robotics*, 10(1):65–92, 2018. https://www.ri.cmu.edu/app/uploads/2020/02/18SORO-BarVel.pdf
2. D. Szafir, B. Mutlu, T. Fong, "Communicating Directionality in Flying Robots," *Proc. 10th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pp. 19–26, 2015. doi:10.1145/2696454.2696475
3. PX4 Autopilot, *LED Meanings (Pixhawk Series)*, PX4 User Guide (main branch). https://docs.px4.io/main/en/getting_started/led_meanings

9. Relation to other algorithms

Visualises modes of Algorithms 6–9 (QUEUED, TAKEOFF, arrival), formation completion of Algorithm 5 and chat activity of Algorithms 12, 14 and 15.

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.